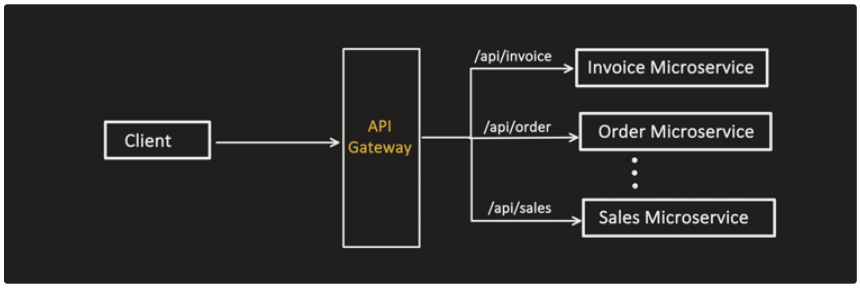
API Gateway - Part1

It provides a Single entry point to access all microservices.



And this Single Entry Point, provides lot of benefits like:

1. Routing : forward requests to right microservices.
2. Load balancing
3. Authentication like JWT
4. Rate Limiting
5. Resilience features (Circuit breaker, Retry etc.)
6. Request/Response Transformation
7. Monitoring and Logging.

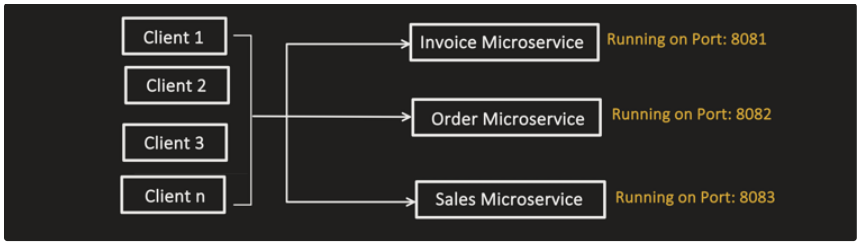
Concept && Coding

Etc..

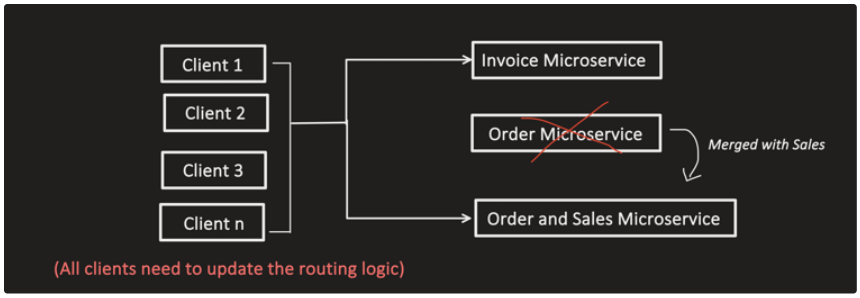
Let's see 1 by 1, how we can achieve all of the above with the API Gateway:

1. **Routing : forward requests to right microservices.**

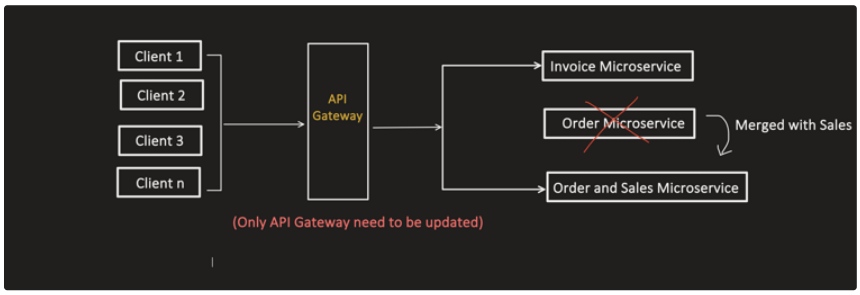
Assume this:



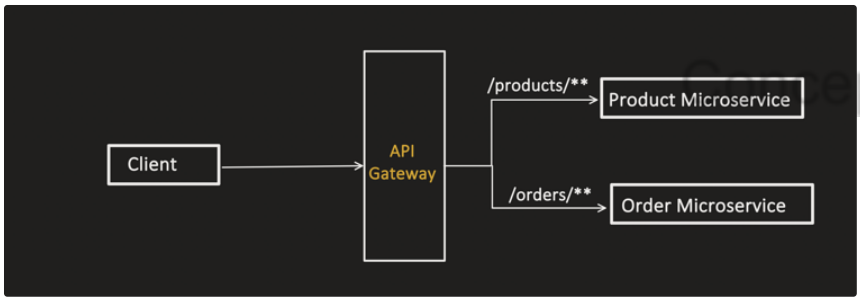
If 1 microservice is further divided into 2 or Multiple microservices merged together, then all clients need to update the routing logic.



But with API Gateway in Mid, changes update required only at 1 service i.e. Gateway



Lets implement this:



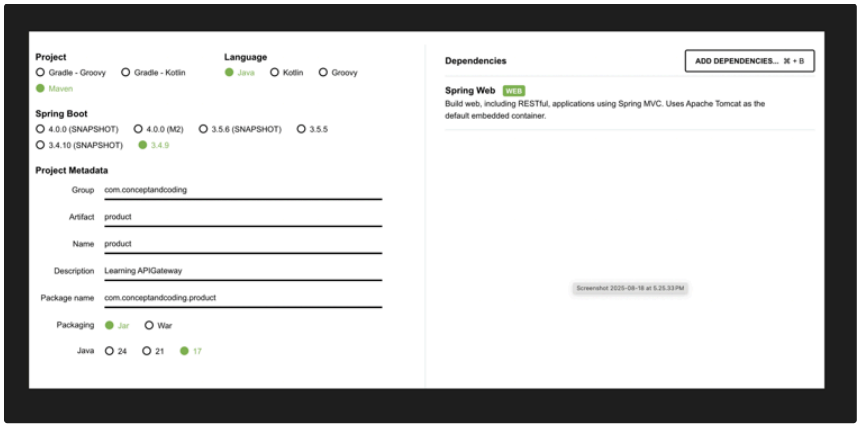
Concept && Coding

<u>Product Controller Class:</u>

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping("/{id}")
    public ResponseEntity<String> getProduct(@PathVariable String id) {
        return ResponseEntity.ok().body("fetch the product details with id:" + id);
    }
}

```

<u>application.properties</u>

```
server.port=8082
spring.application.name=product-service

```



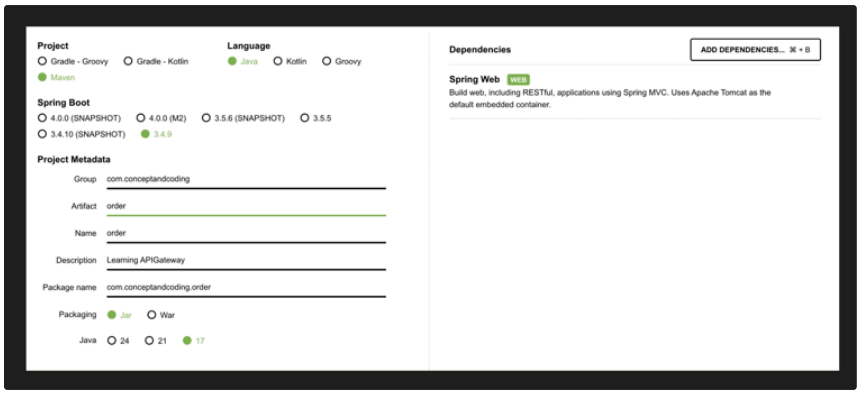
Order Controller Class:

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @GetMapping("/{id}")
    public ResponseEntity<String> getOrder(@PathVariable String id) {
        return ResponseEntity.ok().body("fetch the Order details with id:" + id);
    }
}

```

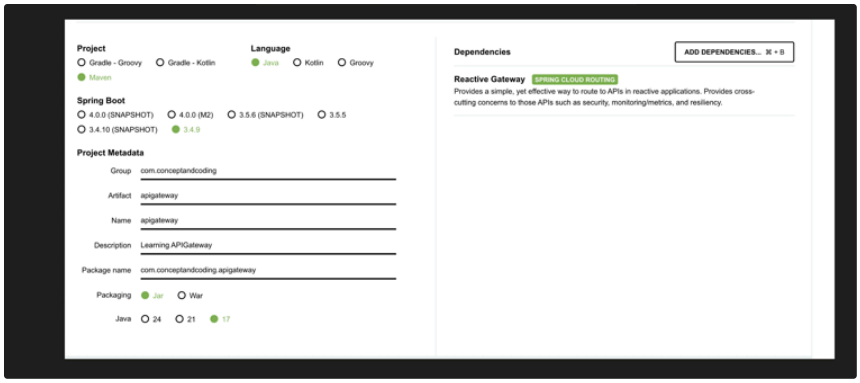
Concept && Coding

application.properties

```
server.port=8081
spring.application.name=order-service

```



Create a new Gateway Application via Spring Initial



In pom.xml, below dependency will get added

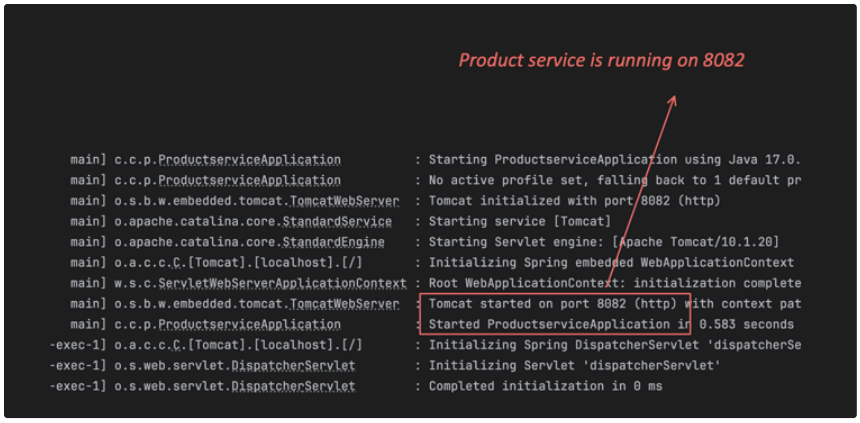
```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>

```

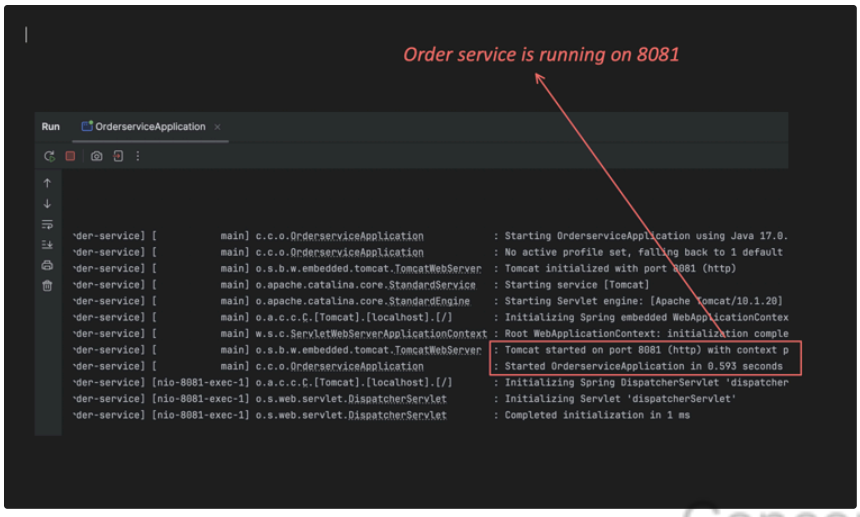
application.properties

```
spring.application.name=apigateway
server.port=8083

# Route: Forward /products/** to product-service
<!--
  Id: This is just a unique name to this route
-->

spring.cloud.gateway.routes[0].id=product-service
<!--
  Uri: Microservice host and port number
-->
spring.cloud.gateway.routes[0].uri=http://localhost:8082
<!--
  predicates: pattern matching to decide the routing.
  apart from Path, we can also have predicate on Method like:
  spring.cloud.gateway.routes[0].predicates[1]=Method=GET,POST

  means GET and POST request to /products/** will match this route.
-->
spring.cloud.gateway.routes[0].predicates[0]=Path=/products/**

# Route: Forward /orders/** to order-service
spring.cloud.gateway.routes[1].id=order-service
spring.cloud.gateway.routes[1].uri=http://localhost:8081
spring.cloud.gateway.routes[1].predicates[0]=Path=/orders/**

```

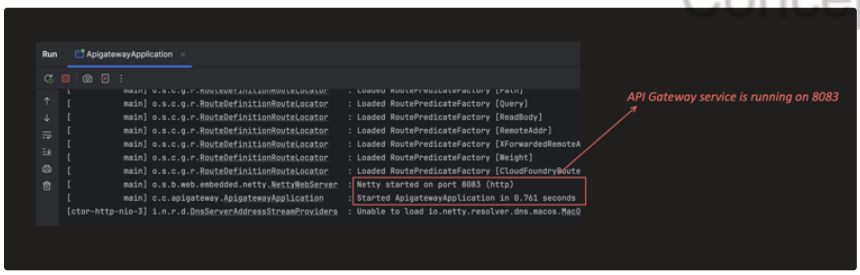
Concept && Coding

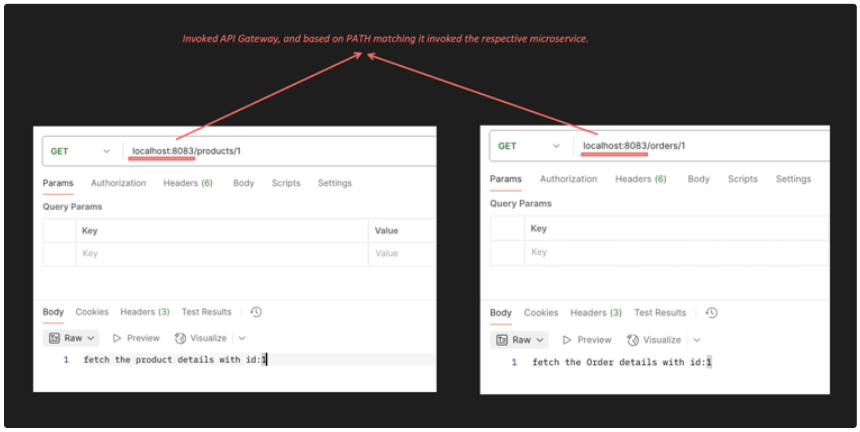
Start all 3 servers: Product, Order and API Gateway



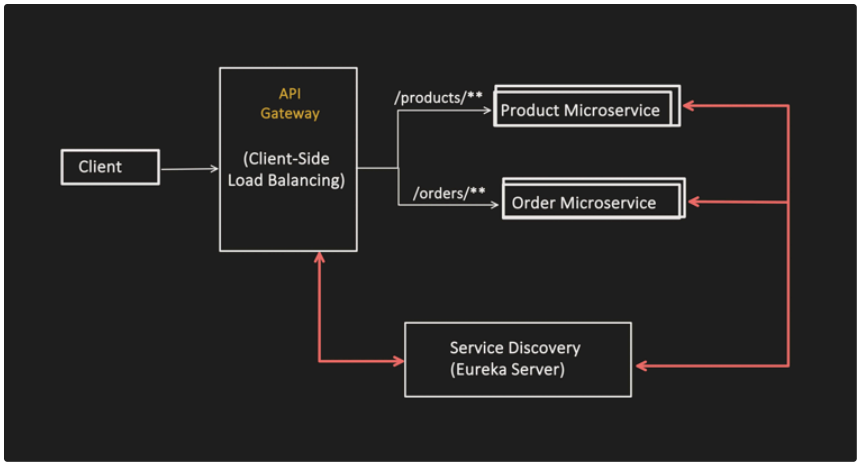


Concept && Coding

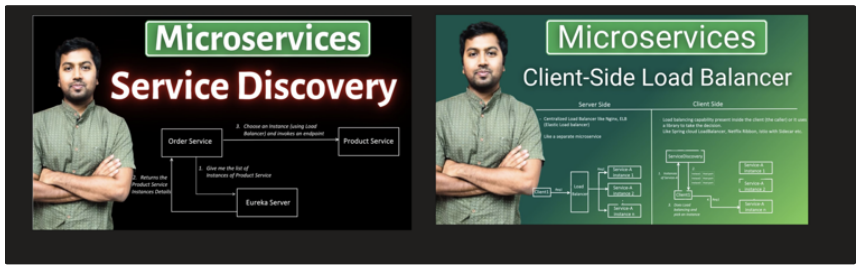




2. **API Gateway and Load Balancer**



*Already covered **Service Discovery** and **Client Side Load Balancer** topic in this playlist, if you have any doubt with below topics, kindly check it out first.*



Concept && Coding

<u>Product Service:</u>

Pom.xml

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

```

application.properties

```
server.port=8082
spring.application.name=product-service

# path of the Eureka Server
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

```

Controller class

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping("/{id}")
    public ResponseEntity<String> getProduct(@PathVariable String id)
{
```

```
        return ResponseEntity.ok().body("fetch the product details with id:" + id);
    }
}

```

Order Service:

Pom.xml

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

```

application.properties

```
server.port=8081
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

```

Controller class

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @GetMapping("/{id}")
    public ResponseEntity<String> getOrder(@PathVariable String id) {
        return ResponseEntity.ok().body("fetch the Order details with id:" + id);
    }
}

```

Concept && Coding

Eureka Server (Service Discovery):

Pom.xml

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>

```

application.properties

```
spring.application.name=eureka-server
server.port=8761

# Since it's a server, we don't want it to register and
# also don't want to fetch the instances details
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false

```

```java
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}

```

API Gateway

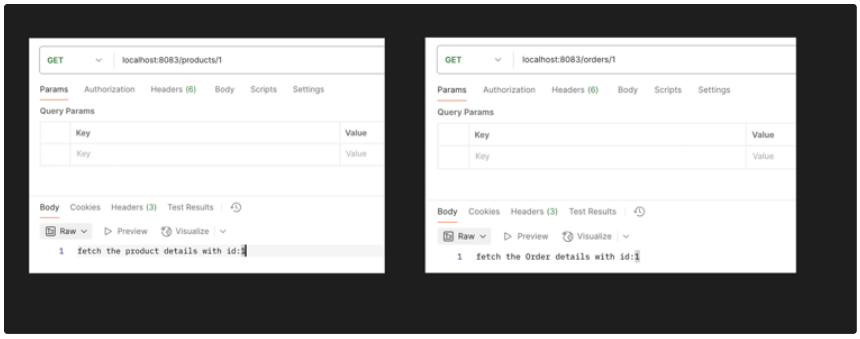
Pom.xml

```pom.xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

```

application.properties

Concept && Coding

```application.properties
spring.application.name=apigateway
server.port=8083

# Route: Forward /products/** to product-service
spring.cloud.gateway.routes[0].id=product-service
<!--
 Replaced the hardcoded URI: http://localhost:8082

  Now internally it will use Spring Cloud LoadBalancer framework to perform the load balancing.
  - first will fetch the instances from Service Discovery and
  -Perform Load balancing algorithm
-->
spring.cloud.gateway.routes[0].uri=lb://product-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/products/**

# Route: Forward /orders/** to order-service
spring.cloud.gateway.routes[1].id=order-service
spring.cloud.gateway.routes[1].uri=lb://order-service
spring.cloud.gateway.routes[1].predicates[0]=Path=/orders/**

# path of the Eureka Server to fetch the microservices host details
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

```



Concept && Coding